Nombre y apellido: _____

Ejercicios (puntaje)			Nota
1 (3 pts)	2 (4 pts)	3 (3 pts)	

CONSIDERACIONES: RESOLVER CADA EJERCICIO EN UNA HOJA DIFERENTE. LOS EJERCICIOS QUE NO ESTÉN CORRECTOS EN UN 50% NO SUMARÁN PUNTOS PARA LA NOTA FINAL. PARA CADA EJERCICIO DEBE **DEFINIR EL TIPO DE DATO** DE CADA TDA UTILIZADO. **EN TODOS LOS CASOS QUE UTILICE ESTRUCTURAS** QUE NO SEAN TDAs "ESTÁNDAR" **DEBE DEFINIR LOS STRUCTS**.

Ejercicio 1: SIN USAR TDA, desarrollar una función (o varias) para **crear un Árbol AVL a partir de un árbol binario de búsqueda**.

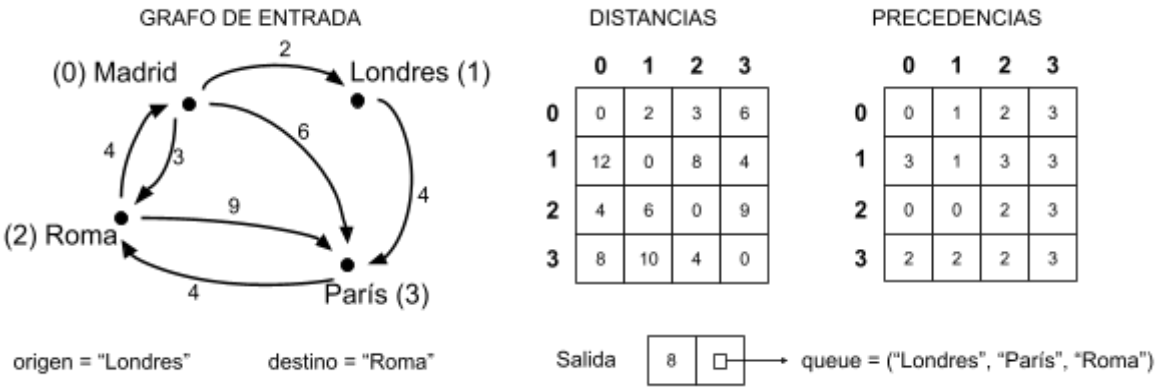
Encabezado de la función principal: `void convertABBToAVL(btn* raiz_ABB, btn** raiz_AVL);`

Se cuenta con la implementación de las siguientes funciones (que no deben ser desarrolladas):

```
btn* createNode(int); //Crea un nodo del árbol binario.
int height (btn*); // Devuelve la altura de un nodo de ab.
int balanceFactor (btn*); // Devuelve el factor de balanceo del nodo.
int leftRotation (btn**); // Rota a izquierda un nodo de ab.
int rightRotation (btn**); // Rota a izquierda un nodo de ab.
```

Ejercicio 2: Dado un **digrafo ponderado** (TDA *Graph*) cuyos vértices son nombres de ciudades, el nombre de una ciudad origen, el nombre de una ciudad destino y las matrices (TDA *Matrix*) devueltas por el algoritmo de Floyd de distancias y precedencias, desarrollar una función (y todas las necesarias) para crear y **devolver en una estructura que contenga un valor entero con el costo minimo de origen a destino y una queue de strings (TDA *queue*) con la secuencia de las ciudades con el camino más corto desde el origen al destino**.

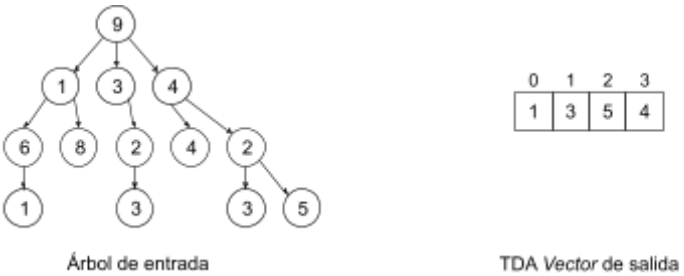
Ejemplo:



Encabezado de la función principal:

```
<definir struct>* get_path (graph* g, char* origen, char* destino,
                            matrix* distancias, matrix* precedencias);
```

Ejercicio 3: Desarrollar una función (y todas las necesarias) para que **dado un árbol n-ario devuelva un TDA *vector* con la cantidad de nodos por nivel**, donde cada índice del vector es el nivel del árbol. Ejemplos:



Encabezado de la función principal: `vector* level_count (ntn* root);`

NOTA: NO debe asumir que el cada nodo contiene calculado el nivel como atributo, si se define como atributo **debe calcularlo**.